面向边缘 AI 大语言模型推理的 RISC-V 自定义 vdot 指令设计——基于“香山”昆明湖 V2 处理器架构

第一阶段竞赛报告

环境验证与 vadd.vv 向量加法流水线分析

面向香山昆明湖 V2 的自定义向量点积指令设计基础

队伍 群贤芯跳

日期 2026 年 11 月 8 日

本阶段围绕 SEW=8 建立软件输入、指令配置、RTL 数据通路和执行波形之间的对应关系，为后续八项 int8 点积指令提供统一的理解基础。

项目	本阶段结果
向量配置	VLEN=128，SEW=8，LMUL=1，AVL=8，实际 vl=8
功能验证	八个 byte 元素自检全部通过，VADD E8 PASS；正常 HIT GOOD TRAP
波形证据	前端握手、译码、重命名、执行结果和 ROB 提交均已按目标 PC 核对
跨阶段衔接	明确区分逐元素加法、乘积归约，以及各自的结果宽度和尾部处理

目录

一 阶段目标与完成情况 3

二 环境搭建与向量配置 4

三 e8 测试设计与程序实现 6

四 波形分析与执行过程 9

五 阶段成果与后续完善 12

一 阶段目标与完成情况

我们完成了香山昆明湖 V2 开发环境核验，并以 SEW=8 的 vadd.vv 建立了软件程序、指令编码、执行波形和 RTL 数据通路的对应关系。第一阶段采用与后续 vdot 相同的源元素宽度，为八项 int8 点积设计提供实验基础。

实现中复用了既有 AM 框架和标准向量加法单元，新增独立的 vadd_e8 程序。测试设置八个活动元素，覆盖正负混合与溢出边界，并通过逐元素软件自检、核级 VCD 和执行状态跟踪完成验证。

评审内容	完成情况	对应证据
环境与过程	保留 VM 清单、AM hello 操作视频及运行日志	已拍操作视频、环境脚本回显和 hello 日志
程序行为	e8/m1、vl=8，八个结果逐项核对	vadd_e8.c、function.log、ELF 反汇编
指令过程	PC=0x80000166、编码 0x021101d7	原始 VCD、前后端 PNG、CSV
数据通路	解释 byte 进位边界、结果寄存和 Mgu 合并	VIAluFix、VIntAdder64b、VIntFixpAlu 源码
阶段衔接	区分 vadd 的逐元素结果与 vdot 的 32 位归约结果	本报告第 2、4 章及第二阶段设计文档

本阶段获得八项正确结果：0、0、0、8、-128、127、127、-50。波形确认目标在采样点 4422 进入执行、4423 输出结果，并于 4428 完成 ROB 提交。我们利用 PC、ROB 和物理目的寄存器标识，将前端处理、算术结果和按序提交关联到同一条动态指令。

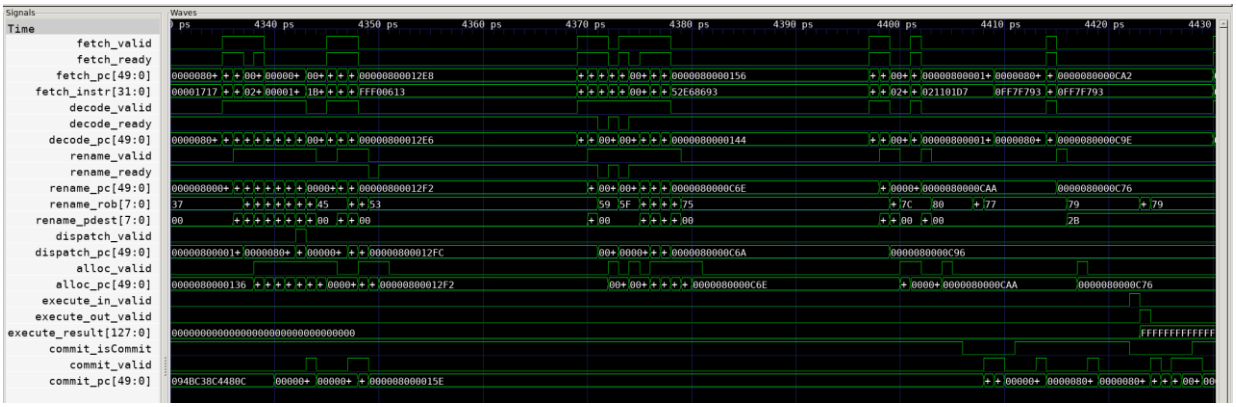


图 1 e8 vadd 从前端到执行与提交的采样窗口

二 环境搭建与向量配置

2.1 开发环境与基础运行

开发与验证工作在 D:\VMs\wyc 的 Ubuntu 环境中完成。e8 vadd 使用 /home/wyc/Desktop/xs-env 下的 AM 框架与香山仿真器，自定义 vdot 使用 /home/wyc/Desktop/vdot-xs 工程。两套工程的路径、运行命令和对应记录均随交付材料保存。

组成	实际使用位置或版本
AM 软件框架	/home/wyc/Desktop/xs-env/nexus-am
e8 测试目录	/home/wyc/vdot_e8_vadd
香山仿真器	/home/wyc/Desktop/xs-env/XiangShan/build/emu
仿真器标识	emu compiled at Oct 22 2026, 10:13:51
构建与观察	riscv64-unknown-linux-gnu-、AM riscv64-xs、GTKWave

视频记录了环境脚本加载、AM 示例清理、rv64gc 程序构建和 emu 镜像加载，并展示 UART 字符串开始输出的画面。hello 的完整输出与结束状态由配套运行日志记录；本次 e8 vadd 另行完成测试和波形采集。

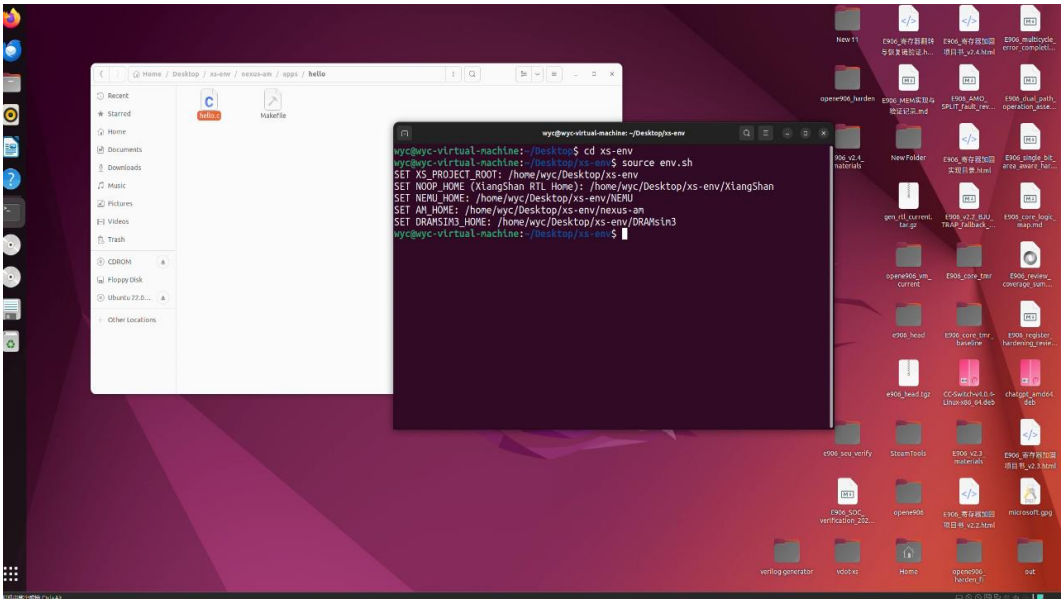


图 2 已有 hello 操作录像中的环境脚本回显

2.2 e8 配置与数据定义

e8 表示 SEW=8，即每个向量元素宽 8 bit。VLEN=128 指单个物理向量寄存器可容纳 128 bit；LMUL 决定一个逻辑向量寄存器组包含多少个这样的寄存器。本例 m1 使用一个寄存器，VLMAX=LMUL×VLEN/SEW=16。软件请求 AVL=8，因此实际 vl=8。

```
vsetvli s4, a5, e8, m1, ta, ma
# a5 = 8; s4 receives vl = 8
vle8.v v1, (a4)
vle8.v v2, (a3)
vadd.vv v3, v1, v2
vse8.v v3, (s1)
```

这次只处理元素 0 至 7；元素 8 至 15 是尾部。本例 vm=1，不读取 v0 作为活动掩码；vstart=0，从第一个元素开始执行。ta 表示尾部不要求保留旧值，ma 表示被屏蔽元素不要求保留旧值；本例没有掩码关闭的活动元素。

属性	本次 e8 vadd	当前 e8 vdot 设计
源元素	低 8 个 byte	低 8 个 byte，按 signed int8 解释
算术	c[i]=(a[i]+b[i]) mod 256	sum(signed8(a[i])×signed8(b[i]))
结果组织	8 个独立的 8 位结果	一个 32 位有符号结果
旧目的数值	不加入活动元素的加法	不作为点积累加输入
高位区域	本次尾部实际为全 1；不作为软件结果断言	vd[127:32] 固定清零

我们按 int8_t 显示逐元素结果，使负数及边界溢出与总线字节一一对应。vadd 的低八位加法结果由位模式决定；后续 vdot 在乘法与归约过程中显式进行有符号解释，形成独立的 32 位结果契约。

2.3 原有多配置样例分析

我们盘点了原工程 apps 下的七个 vadd 样例，归纳出 SEW、LMUL、计数器读取、打印和循环方式的差异。各样例复用同一条标准 vadd.vv；配置写在 vadd.c 中，Makefile 负责选择程序名称和源文件。

目录	SEW/LMUL	有效 vl	计数或循环	源码检查范围
vadd	e64/m1	2	cycle、instret	前两个元素，打印 PASS
vadd_nop	e64/m1	2	无 CSR 计数	首元素，返回差值
vadd_m1_nop	e64/m1	2	v2/v4/v6	首元素，返回差值
vadd_m2_nop	e64/m2	4	v2/v4/v6	首元素，返回差值
vadd_m2_pr	e64/m2	4	打印 vl 和 c[0]	首元素，返回差值
vadd_m2_rd	e64/m2	4	cycle、instret	前两个元素，打印 PASS
vadd_loop	e64/m1	2	重复 10,000 次	打印前两项，无显式 PASS
vadd_e8(新增)	e8/m1	8	独立单次验证	八个元素全检，打印 PASS

依据源码与 VLEN=128，我们计算了旧样例的有效向量长度：AVL 均为 8，e64/m1 的 VLMAX=2、vl=2，e64/m2 的 VLMAX=4、vl=4。本次新增 e8/m1 的 VLMAX=16，实测 vl=8。旧配置表来自源码盘点，本轮动态验证对象为新增 e8 样例。

m2 样例使用 v2/v4/v6 作为三个双寄存器组的起始寄存器，满足组对齐且彼此独立。该配置扩展逻辑向量组容量，元素宽度仍为 64 位。

源码盘点表明，_rd 样例读取 cycle 与 instret，_pr 样例增加结果打印，_loop 样例重复执行 10,000 次；_nop 是原有目录名，其源码中未插入显式 nop 指令。七组源码和 Makefile 副本已一并归档。

我们将配置区分为硬件参数和运行时参数：VLEN 固定为本次处理器的 128 位配置；SEW、LMUL 与 ta/ma 由 vsetvli 设置，AVL 为软件请求数，vl 为实际生效数。现有 RTL 的 e8/e16/e32/e64 分支为扩展测试提供了基础，本阶段集中验证 e8/m1。

三 e8 测试设计与程序实现

3.1 测试数据与边界值

我们设计了八组可人工计算的输入：前三组验证正负抵消，第四组验证普通正数加法，第五、六组覆盖 signed int8 溢出边界，最后两组检查较大正值和负结果。程序同时检查实际 vl 与全部八个输出元素。

i	a[i]	b[i]	数学和	低 8 位	int8_t 结果
0	-1	1	0	00	0
1	2	-2	0	00	0
2	-3	3	0	00	0
3	4	4	8	08	8
4	127	1	128	80	-128
5	-128	-1	-129	7f	127
6	100	27	127	7f	127
7	50	-100	-50	ce	-50

边界用例得到与预期一致的截断结果：127+1 输出 0x80，按 int8_t 解释为 -128；-128-1 输出 0x7f，解释为 127。这两组结果与 -1+1 的抵消用例共同验证了 byte 级加法边界和相邻元素的进位隔离。

```
a bytes [7:0] = 32 64 80 7f 04 fd 02 ff
b bytes [7:0] = 9c 1b ff 01 04 03 fe 01
c bytes [7:0] = ce 7f 7f 80 08 00 00 00
```

我们将软件元素顺序和硬件总线布局进行了对应。十六进制总线按高位在左排列，最右 byte 对应元素 0；内存中的 c[0] 至 c[7] 依次为 00、00、00、08、80、7f、7f、ce。

3.2 测试程序与功能自检

我们采用 int8_t 数组保存输入和输出，在一段内联向量汇编中完成配置、加载、相加和存储，再逐项比较 c 与 expected。全部结果通过后，程序输出 VADD E8 PASS 并以成功返回值结束 AM 运行，仿真日志记录 HIT GOOD TRAP。

```
asm volatile ("li t0, 0x600\n\tcsrs mstatus, t0"
              ::: "t0", "memory");
asm volatile (
  "vsetvli %0, %1, e8, m1, ta, ma\n\t"
  "vle8.v v1, (%2)\n\t"
  "vle8.v v2, (%3)\n\t"
  "vadd.vv v3, v1, v2\n\t"
  "vse8.v v3, (%4)\n\t"
  : "=&r"(vl)
  : "r"((uint64_t)8), "r"(a), "r"(b), "r"(c)
  : "memory"
);
```

程序首先设置 mstatus.VS，随后执行 vsetvli 并检查实际返回的 vl。自检遍历全部八个 c[i]，同时覆盖元素个数、低位运算结果和内存写回结果。

```
int ok = (vl == 8);
for (int i = 0; i < 8; ++i) {
  ok = ok && (c[i] == expected[i]);
}
printf("%s\n", ok ? "VADD E8 PASS" : "VADD E8 FAIL");
return ok ? 0 : 1;
```

```
emu compiled at Oct 22 2026, 10:13:51
Using simulated 32768B flash
Using simulated 256MB RAM
The image is /home/wyc/vdot_e8_vadd/build/vadd_e8-riscv64-xs.bin
SEW=e8 vl=8
vadd result: 0 0 0 8 -128 127 127 -50
VADD E8 PASS
Core 0: HIT GOOD TRAP at pc = 0x80000220
Core-0 instrCnt = 4,572, cycleCnt = 13,812, IPC = 0.331017
```

图 3 e8 vadd 实际运行日志及成功结束状态

本次完整程序记录 instrCnt=4,572、cycleCnt=13,812，包含启动、访存、打印和退出流程。我们另从执行接口波形测得单元输入输出间隔，将整程序统计与执行通路分析分别记录。

3.3 编码核验与译码关联

我们通过 ELF 反汇编将目标定位为 PC=0x80000166，机器码为 0x021101d7。vsetvli 设置的 e8 状态经 vtype 传递到执行单元，使同一条标准 vadd 编码在本次运行中按 8 位元素执行。

```
8000015a: 0c07fa57  vsetvli s4,a5,e8,m1,ta,ma
8000015e: 02070087  vle8.v v1,(a4)
80000162: 02068107  vle8.v v2,(a3)
80000166: 021101d7  vadd.vv v3,v1,v2
8000016a: 020481a7  vse8.v v3,(s1)
```

字段	位段	本次值	意义
funct6	31:26	000000	向量加法
vm	25	1	无 v0 掩码
vs2	24:20	00001	v1，即数组 a
vs1	19:15	00010	v2，即数组 b
funct3	14:12	000	OPIVV
vd	11:7	00011	目的 v3
opcode	6:0	1010111	OP-V

```
VADD_VV -> OPIVV(
  FuType.vialuF, VialuFixType.vadd_vv, T, F, F
)
```

VecDecoder 把该编码映射到向量整数 ALU 和 vadd_vv 操作类型；执行时再把 vsew 传入类型与进位控制。波形中目标操作的 fuOpType=0x040、vsew=0，分别对应本构建的加法操作和 e8 配置。

我们按 vd、vs2、vs1 的汇编顺序核对了寄存器字段：vadd.vv v3,v1,v2 对应目的 v3、vs2=v1、vs1=v2。该字段关系与源数组及执行接口上的数据相符。

四 波形分析与执行过程

4.1 前端握手与微操作关联

我们从核级 VCD 中提取目标 PC、指令编码及握手信号。前端在 4338 同时满足 valid=1 和 ready=1，完成本条指令接收；此后按相同 PC 跟踪译码和重命名状态。

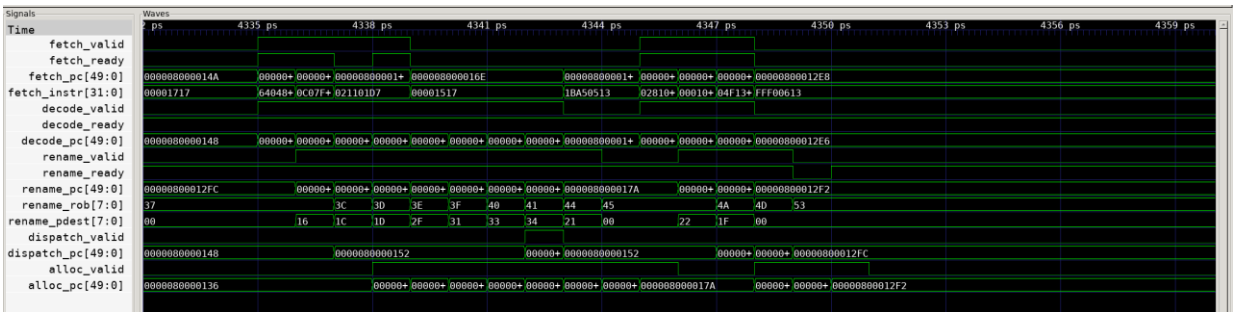


图 4 e8 目标指令的前端握手、译码、重命名和分配

采样点	确认的事件	关联标识
4338	前端 valid=1 且 ready=1	PC=0x80000166，instr=0x021101d7
4340	decode 接口握手有效	相同 PC 与指令
4341	rename 接口握手有效	ROB=0x40，pdest=0x33
4342	vfUops[2] 派发有效	目标 PC
4343	ROB enq[0] 记录有效	目标 PC

重命名阶段为目标分配 ROB=0x40 和物理目的寄存器 0x33。执行输出再次携带同一组标识，完成了前端分配与后端结果的动态关联。

4.2 执行结果与按序提交



图 5 e8 执行入口、byte 结果与按序提交

采样点	有效条件或内容	结果解释
4422	execute_in_valid=1，ready=1；op=0x040，vsew=0	接受 e8 加法的两个源向量
4423	execute_out_valid=1，ready=1；ROB=0x40，pdest=0x33	目标结果形成，与重命名关联
4423	res[63:0]=0xce7f7f8008000000	低 8 byte 与软件自检一致
4423	Mgu vl=8、vstart=0、ta=1	高 8 byte 属于尾部
4427	目标 PC 与 commitValid=1，但 isCommit=0	不是本次指令的真实提交
4428	isCommit=1 且 commitValid=1；目标 PC/编码一致	ROB 按序提交目标指令

```
src0 = ffffffffffffffff_9c1bff010403fe01
src1 = ffffffffffffffff_3264807f04fd02ff
res  = ffffffffffffffff_ce7f7f8008000000
```

我们核对了两个输入的 byte 布局：源 0 低 64 位对应数组 b，源 1 低 64 位对应数组 a。执行结果的低八个 byte 与软件自检完全一致。输入接收与输出有效相隔一个记录周期，输出后总线保持该结果。

本次 VCD 以 1 ps 为时间标记单位，并按仿真步记录快照；所选 clock 信号在记录中保持为 1。本文采用采样点之间的相对间隔描述过程，硬件频率与物理时序属于后续综合分析范围。

4.3 byte 加法数据通路

我们对照 VIAluFix 源码梳理了 e8 加法通路：128 位执行接口复用两个 64 位运算模块，每个模块内部按 byte 组织加法，并由 SEW 控制进位边界。

```
private val dataWidth = cfg.destDataBits
private val dataWidthOfDataModule = 64
private val numVecModule = dataWidth / dataWidthOfDataModule
private val vIntFixpAlus = Seq.fill(numVecModule)(
  Module(new VIntFixpAlu64b))
```

在本构建 dataWidth=128，因此 numVecModule=2。普通非拓宽加法按相应数据分组送入模块，最终由 Cat 重新拼接。八个活动元素恰好位于低 64 位，高 64 位虽有算术通路，仍由 Mgu 的 vl/ta 规则决定最终有效写回。

```
if (i == 0) {
  cin(i) := carryIn(i)
} else if (i == 4) {
  cin(i) := Mux(eewCin.is64, cout(i-1), carryIn(i))
} else if (i % 2 == 0) {
  cin(i) := Mux(eewCin.is64 || eewCin.is32,
                cout(i-1), carryIn(i))
} else {
  cin(i) := Mux(eewCin.is8, carryIn(i), cout(i-1))
}
```

VIntAdder64b 的循环变量 i 是 byte 编号。在 e8 下，上述分支均为每个 byte 选择自己的 carryIn；对于本例普通 vadd，它为 0，相邻 byte 的 cout 不传递。这直接解释元素 0 的 -1+1 得到 0 后，不会把进位加到元素 1。

```
val vdAdderS1 = RegEnable(vIntAdder64b.io.vd, fire)
io.vd := Mux(isFixpS1, vFixPoint64b.io.vd,
             Mux(isMiscS1, vdMiscS1, vdAdderS1))
```

源码中的结果寄存器与波形中 4422 接收、4423 输出的间隔一致。我们据此建立了进位控制、结果寄存与实际动态执行之间的对应关系，完整参考源码随测试材料交付。

4.4 尾部合并与旧目的值

我们进一步分析了向量框架的 oldVd 合并路径。该路径为保留策略提供旧数据，活动元素的 vadd 运算仍由两个显式源操作数完成。Mgu 结合 vl、vstart、mask 和尾部策略形成最终写回值。

```
private val oldVdSplit = Module(
  new VecDataSplitModule(dataWidth, dataWidthOfDataModule))
oldVdSplit.io.inVecData := oldVd

mgu.io.in.info.vl := Mux(outIsVmvsx, 1.U, outVl)
mgu.io.in.info.vstart := outVecCtrl.vstart
mgu.io.in.info.eew := outEew
mgu.io.in.info.vsew := outVecCtrl.vsew
```

区域或选项	含义	本次观测
活动元素 0—7	执行两个源的逐元素加法	低 64 位为 ce7f7f8008000000
尾部元素 8—15	vl=8，不属于活动结果	ta=1 时本实现实际写入全 1
掩码 vm=1	本次不使用 v0 屏蔽活动元素	无被掩码关闭的活动 byte
旧目的寄存器	保留策略的数据来源	不加入本例活动元素的算术和

本次完整向量结果为 0xffffffffffffffff_ce7f7f8008000000：低 64 位保存八个活动结果，高 64 位在 ta=1 下实际写入全 1。软件通过 vse8.v 存储并检查八个活动元素，尾部区域按既定策略处理。

源与目的重叠可表示为 vadd.vv v3,v3,v1，此时旧 v3 作为一个显式源参与相加。本次验证采用独立源寄存器，重点检查配置、逐元素计算及写回过程。

与标准 vadd 的逐元素写回相衔接，我们在第二阶段为 vdot 定义了八项有符号乘积归约：结果写入 vd[31:0]，高 96 位清零，算术输入为两个源向量。该定义与本次 byte 数据布局共同构成跨阶段的实现基础。

五 阶段成果与后续完善

我们完成了 e8 标准向量加法的程序开发、八项结果自检、边界值检查及核级流水线分析。交付材料包括 Word 报告、原始 VCD、逐点 CSV、GTKWave 图片、完整测试源码和录制指导，并保留了原有多配置样例的源码记录。

材料	提交位置
主报告	阶段 1/report/01_第一阶段_环境验证与流水线分析报告.docx
e8 GTKWave 与终端 PNG	阶段 1/png/vadd_e8_frontend.png、vadd_e8_backend.png 等
VCD 与逐点采样	阶段 1/report/执行日志/vadd_e8_3500_5000.vcd.gz、vadd_e8_stage_samples.csv
源码与反汇编	coding/phase1_vadd_e8/vadd_e8.c、Makefile、vadd_e8.dis
波形查看脚本与映射	coding/phase1_vadd_e8/*.tcl、signal_aliases.json
多配置完整副本	coding/phase1_vadd_e8/original_configurations
操作视频与详细补录指导	阶段 1/video/操作视频.mp4、第一阶段_操作视频说明与终端命令.docx

本阶段实测条件为 e8/m1、vl=8、vm=1、vstart=0。我们记录了执行输入输出间隔及提交位置；其他向量参数组合、掩码与重启状态，以及队列等待的细分分析列为后续完善方向。

我们同时保存完整层级 VCD 与信号筛选版 focus.vcd。筛选版保留原始时间戳和信号值，signal_aliases.json 记录路径映射，CSV 提供逐采样点数值，形成了可复核的波形分析材料。

通过本阶段工作，我们掌握了 e8 数据组织、有效元素范围、向量配置和既有执行通路的关键关系。后续 vdot 在此基础上增加有符号乘法与归约，采用低 32 位结果和高位清零的自定义写回格式。